



ASSESSMENT TOOL 2 - Technical Presentation

COURSE NAME: Data Structure

COURSE CODE: CSA03

Assessment Tool Weightage: 30%

COURSE OUTCOME COVERED

CO1: Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL3, BL4)

Questions: EACH QUESTIONS CARRIES : 10 MARKS

Duration :ONE HOUR

S.No	List of Questions
1	A student management system needs to store the marks of 1000 students efficiently. Apply the concepts of primitive and non-primitive data structures to select an appropriate structure for storing and managing the data and prepare a technical presentation justifying your choice based on the efficiency of data structure.
2	Apply a suitable data structure to implement an LRU cache system. Prepare a technical presentation explaining how it improves performance.
3	Apply suitable data structure for the application that encourages passengers to book tickets online and how the requests are processed in order of arrival. Prepare a technical presentation justifying its efficiency using complexity analysis.
4	Choose a suitable linear or non-linear data structure for a navigation system that stores city connections and finds shortest routes between locations. Prepare a technical presentation explaining why it is appropriate.
5	A web browser supports navigating backward through previously visited pages. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency.
6	A digital library allows searching books using ISBN numbers quickly among 50,000 records. Select the most suitable data structure and Prepare a technical presentation justifying using time complexity comparison.
7	A social networking platform stores friend relationships between users. Recommend a suitable representation structure and Prepare a technical presentation explaining why it supports efficient relationship mapping.
8	Customer service requests are processed in the order they arrive. Identify the suitable ADT and Prepare a technical presentation explaining why it ensures fair request processing.
9	A programmer uses recursion to compute factorial values. Explain how recursion internally uses a data structure and analyze its space complexity implications. Prepare a technical presentation.



10	An institute frequently sorts student marks before publishing rank lists. Suggest a suitable data structure for storing the records and justify your selection based on sorting efficiency considerations. Prepare a technical presentation
11	A hospital emergency ward treats patients based on severity rather than arrival order. Suggest a suitable data structure for managing patients and prepare a technical presentation explaining your choice.
12	A music streaming application maintains a playlist where songs can be added, removed, or reordered dynamically. Recommend a suitable data structure and justify based on flexibility and efficiency. Prepare a technical Presentation
13	A dictionary application needs to quickly search whether a word exists in its database. Suggest an appropriate data structure and justify based on search performance. Prepare a technical Presentation.
14	An operating system manages multiple processes waiting for CPU allocation in a time-sharing environment. Recommend an appropriate data structure and justify based on time and space efficiency.
15	An e-commerce website stores product information and frequently updates stock availability. Recommend a suitable ADT and prepare a technical presentation explaining how it improves performance efficiency.
16	A printer processes multiple print requests in the order they are received. Recommend an appropriate data structure and prepare the presentation explaining your choice.
17	A spell-checking application verifies whether typed words exist in its dictionary database. Suggest a suitable data structure and justify based on search speed.
18	A metro rail network maps stations and routes to determine reachable stations from a source station. Suggest a suitable data structure for representation and justify your answer.
19	A compiler checks whether parentheses in a program are balanced during syntax analysis. Identify the suitable data structure and justify based on operational requirements.
20	An online leaderboard system continuously updates player rankings after each match. Suggest an efficient data structure and justify based on sorting and update efficiency.
21	A task scheduling system assigns jobs based on execution priority levels. Suggest a suitable data structure and justify performance efficiency.
22	A maze-solving algorithm explores multiple paths recursively. Recommend the underlying structure and justify its efficiency.
23	A contact management application frequently inserts and deletes user records dynamically. Suggest a suitable data structure and justify your selection.
24	A banking system retrieves customer account details using account numbers instantly. Suggest an



	appropriate data structure and justify your selection.
25	A university stores student registration numbers and frequently searches them during examinations. Recommend a suitable structure and justify based on search efficiency.
26	A login authentication system verifies usernames quickly among millions of users. Suggest a suitable data structure and justify your answer.
27	A road traffic monitoring system identifies alternate paths during congestion. Recommend a suitable structure and justify using complexity reasoning.
28	A file system organizes directories and subdirectories hierarchically. Suggest a suitable non-linear data structure and justify your selection.
29	A database indexing system needs fast insertion, deletion, and searching operations. Recommend an appropriate tree data structure and justify your choice
30	A decision-making application classifies outcomes based on multiple conditions. Suggest a suitable data structure and justify its effectiveness.
31	An university stores course prerequisites forming hierarchical dependencies. Recommend an appropriate representation structure and justify its efficiency.
32	A ticket counter processes passengers waiting in line for boarding passes. Suggest an efficient data structure and explain why it is appropriate.
33	A browser maintains forward navigation after visiting multiple web pages. Recommend an appropriate data structure and justify your choice.
34	A program converts infix expressions into postfix expressions during compilation. Identify the suitable structure and justify its efficiency.
35	A syntax checker verifies matching HTML tags in a web page. Suggest a suitable ADT and justify its efficiency.
36	A calculator application evaluates arithmetic expressions entered by users. Recommend a suitable data structure and justify based on expression processing efficiency
37	A university attendance system records student presence sequentially during each class hour. Recommend a suitable data structure and justify your choice based on insertion efficiency.
38	A railway reservation waiting list updates passenger positions dynamically as cancellations occur. Suggest a suitable data structure and justify your selection with complexity analysis.
39	A messaging application stores recent chat messages temporarily for quick access. Recommend an appropriate data structure and explain its efficiency.
40	A cafeteria token system serves customers strictly in the order of arrival. Identify the suitable ADT and justify why it ensures fairness.

Code of Conduct:

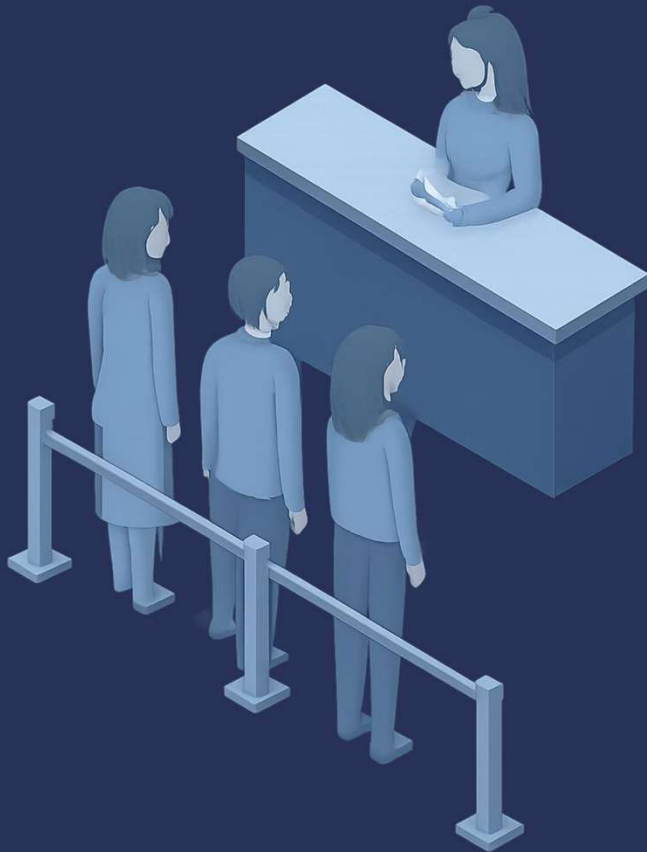
*I **Deepak R (192511008)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

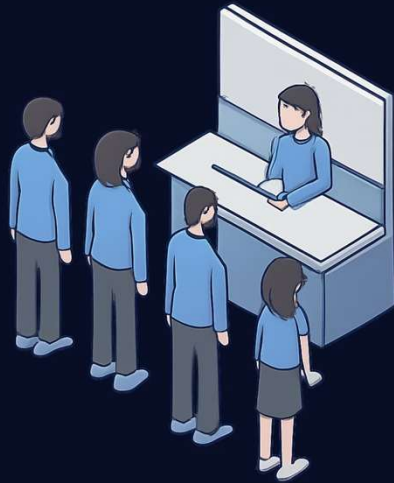
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness



Queue ADT for Fair Customer Service Request Processing

Deepak.R | Register No: 192511008 | Data Structures (CSA03) –
Technical Presentation | SIMATS Engineering

Introduction



→ Customer Service

Managing and resolving requests submitted by customers efficiently and fairly

→ The Problem

Random handling leads to skipped requests, unfair delays, and poor customer experience

→ The Need

A structured system that preserves arrival order and ensures every request is served

→ Solution

Queue ADT – a linear data structure that enforces **First In, First Out (FIFO)** order

Problem Statement

A customer support center receives **hundreds of service requests** every day. The system must handle them systematically.

Accept

Receive requests continuously as they arrive

Process

Handle requests fairly – no skipping, no priority bias

Maintain

Strict order of arrival at all times

Scale

Handle high volumes without breaking down



Queue ADT — The Right Data Structure

FIFO Principle: First In, First Out – the first request added is the first to be processed. New requests enter at the **Rear**; processed requests exit from the **Front**.

Queue Structure: Front → Rear

Front			Rear
[Req 1]	[Req 2]	[Req 3]	[Req 4]

Queue Operations

Operation	Description
Enqueue	Insert at Rear
Dequeue	Remove from Front
Peek	View Front element
IsEmpty	Check if queue is empty

How the Queue Works

01

Customer Request

Customer submits a service request to the system

02

Enqueue

Request is added to the **Rear** of the queue

03

Queue Holds Requests

All pending requests wait in strict FIFO order

04

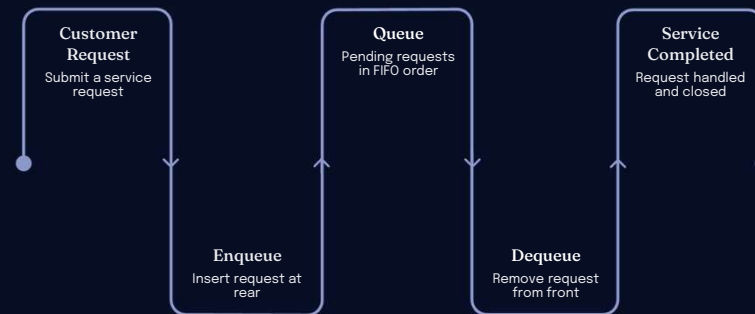
Dequeue

Service rep picks the **Front** request and removes it

05

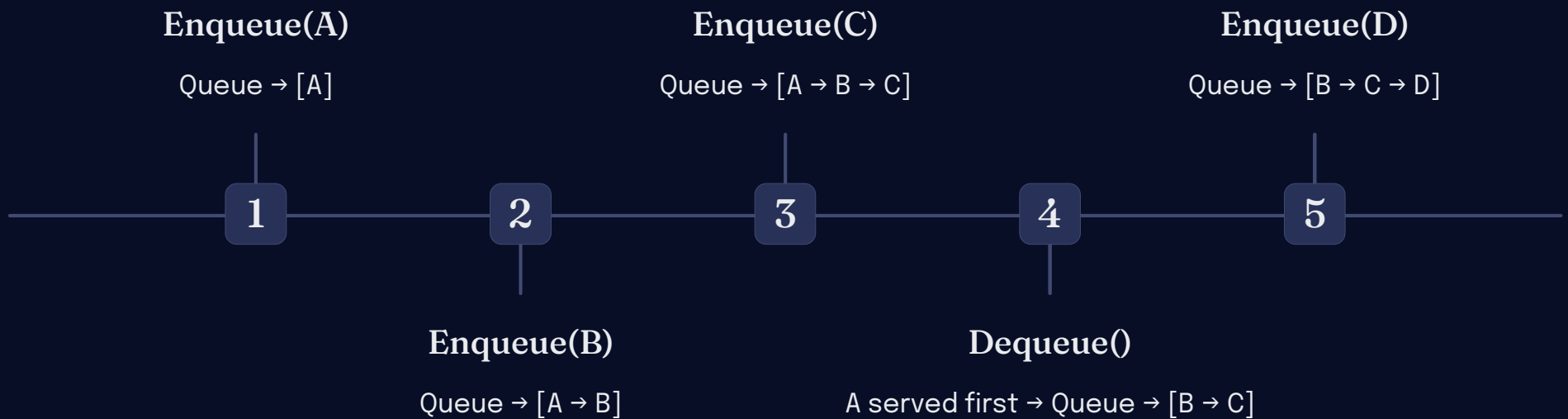
Service Completed

Request is resolved; next request becomes the new Front



Step-by-Step Example

Tracing the Queue through a series of Enqueue and Dequeue operations – **fairness guaranteed by FIFO**.



✓ ☒ Customer A is always served first – **fairness guaranteed** by FIFO.

Time & Space Complexity

Complexity Table

Operation	Time Complexity
Enqueue	$O(1)$
Dequeue	$O(1)$
Peek	$O(1)$
IsEmpty	$O(1)$
Space	$O(n)$

$O(1)$

All Core Operations

Enqueue, Dequeue, Peek, and IsEmpty all run in constant time – no traversal needed

$O(n)$

Space Complexity

Memory grows linearly with the number of requests stored in the queue

Real-World Applications

Queue ADT is the backbone of ordered, fair processing across many real-world systems.



Customer Service Centers

Ticket and chat request queues handled in arrival order



Call Centers

Callers held in order until a free agent is available



Bank Token Systems

Numbered tokens served strictly in sequence



Printer Spooling

Print jobs queued and executed sequentially



CPU Scheduling

Processes managed in FIFO order by the operating system



Railway Reservation

Booking requests processed in strict arrival order

Advantages & Limitations

✓ Advantages

Fair FIFO Processing

No request is skipped or starved – arrival order always respected

Fast $O(1)$ Operations

Efficient insertion and deletion with no traversal overhead

Simple to Implement

Suitable for real-time systems with predictable behavior

⚠ Limitations

No Random Access

Cannot reach middle elements directly – only Front and Rear accessible

Fixed-Size Overflow

Array-based queues may run out of space under heavy load

No Priority Handling

Urgent requests cannot jump ahead – a Priority Queue is needed instead

Conclusion

FIFO Principle

Every customer is served in arrival order – fairness is built into the structure

O(1) Efficiency

Enqueue, Dequeue, Peek, and IsEmpty all run in constant time

Widely Applied

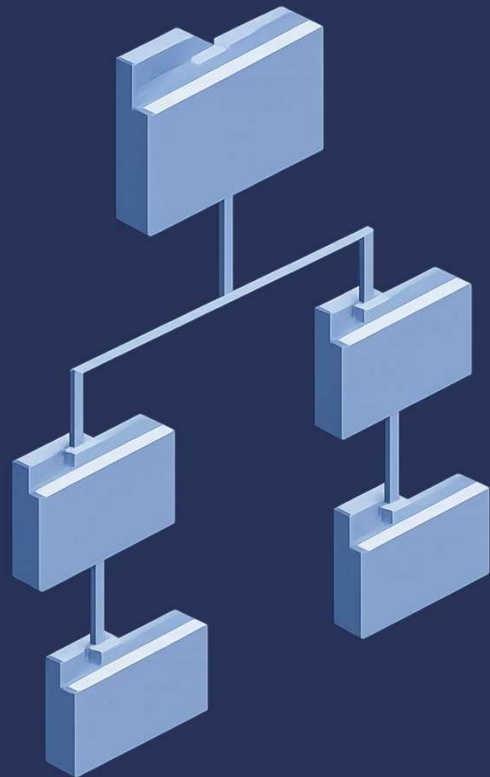
Customer support, banking, CPU scheduling, and printer management

Best Fit

Queue ADT is the most suitable structure for fair, ordered service request processing

Thank You | Questions?

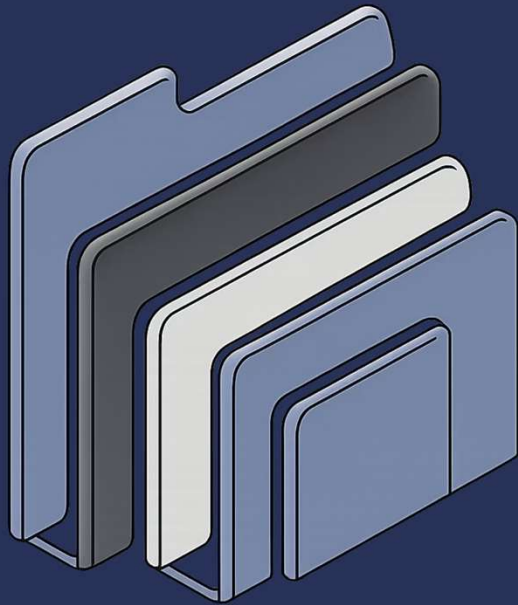




Tree Data Structure for File System Organization

Deepak R | Reg. No.: 192511008

Data Structures (CSA03) – SIMATS Engineering



What is a File System?

File System

A method for organizing, storing, and retrieving files on a storage device

The Challenge

Modern computers store thousands of files – flat lists become completely unmanageable

The Solution

Hierarchical organization using a **Tree Data Structure** – mirrors how folders nest inside folders

Tree Advantage

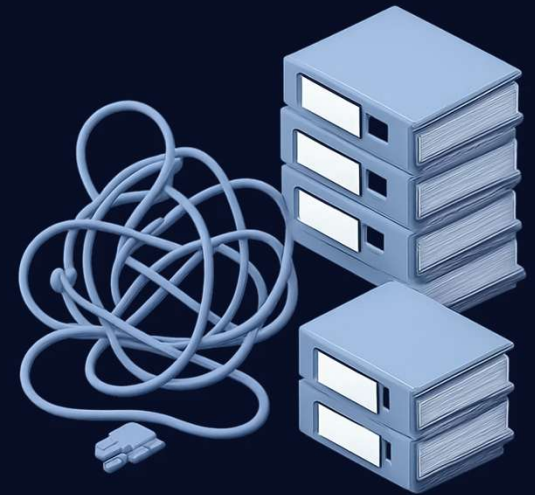
Parent-child relationships naturally model **Root → Folders → Subfolders → Files**

Problem Statement

Scenario: A computer stores thousands of files across hundreds of folders. The system must meet these requirements:

- Organize files **systematically** with clear parent-child relationships
- Support **quick navigation** from root to any file
- Allow **creation, deletion, and retrieval** of subfolders and files efficiently

❏ A flat array or linked list cannot model this – a **non-linear Tree structure** is required.



Why a Tree? — The Right Data Structure



Root Node

Represents the main drive (C:\, D:\)



Internal Nodes

Represent folders and subfolders



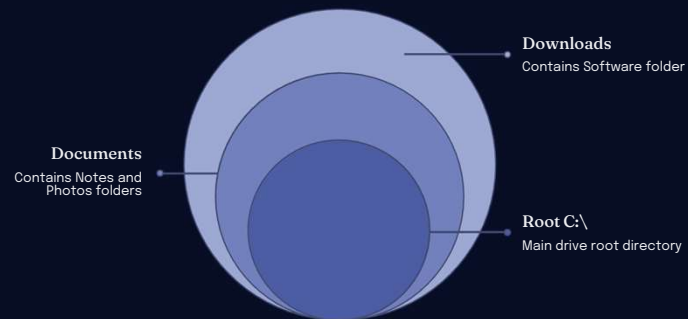
Leaf Nodes

Represent individual files



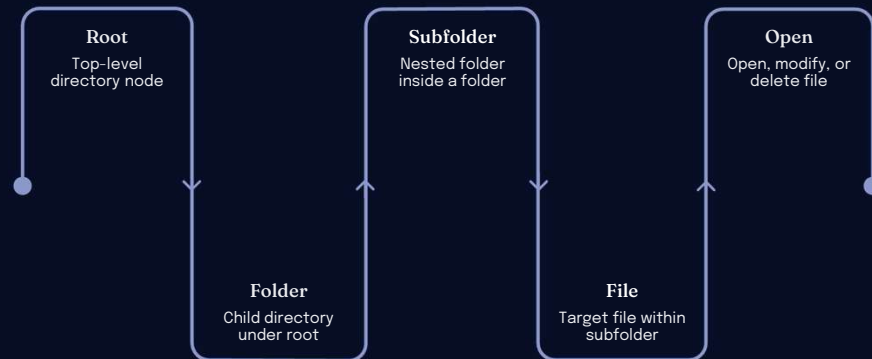
Edges

Parent-child links between directories



Blue folder nodes branch from the root; green leaf nodes represent final files and folders with no further children.

How the File System Tree Works



Navigation always follows a top-down path – from the root through nested folders until the target file is reached.

Step-by-Step Navigation

01

Root Directory

Created as the top-level node of the entire tree

02

Folders as Child Nodes

Folders are added as child nodes of the root

03

Nested Subfolders

Subfolders are created inside folders as nested children

04

Files as Leaf Nodes

Files are stored as leaf nodes – no further children

Real-World Example — C:\ Drive

Directory Path



Tree Breakdown

- **Root:** C:\ – top of the entire directory tree
- **Internal nodes:** Users, Deepak, Documents, Pictures, Downloads, Program Files
- **Leaf nodes (files):** Resume.pdf, Notes.docx
- **Full path:** C:\ → Users → Deepak → Documents → Resume.pdf

Time & Space Complexity

Key Variables

- **h** = height of the tree (depth of folder hierarchy)
- **n** = total number of nodes (files + folders)
- Search time depends on **how deep** the target file is nested

ⓘ Most operations run in **$O(h)$** time – making deep, balanced trees highly efficient for large file systems.

Operation	Time Complexity	Notes
Insert Folder	$O(h)$	Traverse to parent node
Delete Folder	$O(h)$	Locate then remove subtree
Search File	$O(h)$	Depth-first path traversal
Traverse Tree	$O(n)$	Visit all nodes once
Space	$O(n)$	One node per file/folder

Real-World Applications



Desktop OS

Windows (NTFS), Linux, and macOS Finder all use tree-based directory models for file organization



Cloud Storage

Google Drive, OneDrive, and Dropbox implement virtual folder trees stored and synced in the cloud



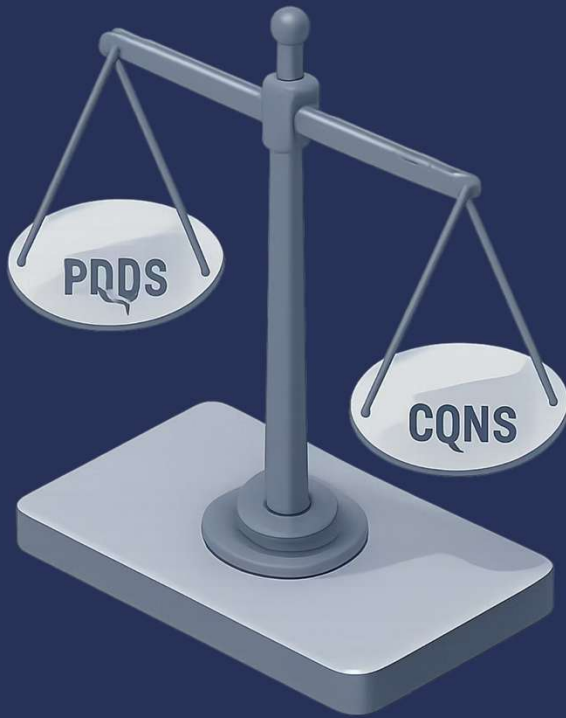
Mobile & Embedded

Android File Manager and IoT device storage systems rely on tree structures for compact organization



Databases & Compilers

B-Trees power database indexing; Abstract Syntax Trees enable code parsing in compilers



Advantages & Limitations

✓ Advantages

- **Hierarchical organization:** Natural parent-child model for directories
- **Efficient navigation:** $O(h)$ search – no need to scan all files
- **Scalable:** Supports millions of files without restructuring

✗ Limitations

- **Deep trees:** Very deep hierarchies slow down search and traversal
- **Complex deletion:** Removing a folder requires deleting its entire subtree
- **Memory overhead:** Each node stores pointers – more memory than flat arrays

Conclusion

Best-Fit Structure

A **Tree** is the most suitable non-linear data structure for file system organization

Hierarchical Storage

Provides natural parent-child relationships – **Root** → **Folders** → **Files**

Efficient Operations

Enables fast **insert**, **delete**, **search**, and **traversal** across the entire directory

Universal Adoption

Used by **Windows**, **Linux**, **macOS**, and all major cloud platforms worldwide

Thank You – Questions?

